



Senate Building Decoration (decoration)

Du bist beteiligt am Bau eines grossen neuen Senatgebäudes, und eine wichtige Aufgabe dabei ist es, eine zentrale Dekoration auszuwählen. Der Senat will die teuerste mögliche Dekoration. Allerdings möchten die Plebianer auch Teil der Auswahl sein, und sie sind besorgt, dass eine sehr teure Dekoration zu höheren Steuern führen könnte. Folglich wollen sie die billigste mögliche Dekoration aussuchen.

Zur Entscheidung wird eine Reihe an N möglichen Dekorationen präsentiert, welche in einer Linie angeordnet sind. Jede Dekoration, von 0 bis $N - 1$ indiziert, hat einen bestimmten Preis, bezeichnet mit A_i .

Nach langen Diskussionen wurde folgende Entscheidungsmethode gewählt. Der Senat und die Plebianer wechseln sich ab im Auswählen. **Der Senat fängt an.** Ihre Ziele sind gegensätzlich:

- Der Senat möchte sicherstellen, dass die endgültige Dekoration den **grösstmöglichen** Preis hat.
- Die Plebianer möchten sicherstellen, dass die endgültige Dekoration den **kleinstmöglichen** Preis hat.

In jedem Zug berücksichtigt der derzeitige Spieler (Senat oder Plebianer) die Dekorationen, welche noch verfügbar sind. Er muss einen kontinuierlichen Block (ein Subarray) jener Dekorationen auswählen; der ausgewählte Block darf **nicht der ganze momentane verfügbare Block** sein und muss **mindestens halb so lang** (aufgerundet) sein wie der momentane Dekorations-Block. Alle Dekorationen, welche **ausserhalb** des gewählten Blockes liegen, werden dann entfernt.

Dieser Prozess geht weiter, wobei die Reihe an Dekorationen nach jedem Zug schrumpft, bis nur eine einzige Dekoration übrig bleibt. Der Preis dieser endgültigen Dekoration ist das Resultat. Deine Aufgabe ist es, diesen Preis vorherzusagen, angenommen sowohl der Senat als auch die Plebianer spielen optimal, um ihre jeweiligen Ziele zu erreichen.

Implementierung

Du musst eine einzelne `.cpp` Quelldatei einreichen.



Unter den Anhängen zu dieser Aufgabe findest Du eine Vorlage `decoration.cpp` mit einer Beispielimplementierung.

Du musst die folgende Funktion implementieren:

C++

```
int decorate(int N, vector<int> A)
```

- Die Ganzzahl N stellt die anfängliche Anzahl der möglichen Dekorationen dar.
- Das Array A , indiziert von 0 bis $N - 1$, enthält die Werte $A_0, A_1, \dots, A_{N-1}$, wobei A_i der Preis der i -ten Dekoration ist.
- Die Funktion soll den Preis der ausgewählten Dekoration zurückgeben.

Beispielgrader

Das Verzeichnis der Aufgabe enthält eine vereinfachte Version des Jury-Graders, die du verwenden kannst, um deine Lösung lokal zu testen. Der vereinfachte Grader liest die Eingangsdaten aus der

Datei `stdin`, ruft die Funktion `decorate` auf und schreibt schliesslich die Ausgabe in die Datei `stdout` mit folgendem Format.

Die Eingabe besteht aus 2 Zeilen, die Folgendes enthalten:

- Zeile 1: die Ganzzahl N .
- Zeile 2: die durch Leerzeichen getrennten Ganzzahlen, welche die Elemente $A_0, A_1, \dots, A_{N-1}$ darstellen.

Die Ausgabe besteht aus einer einzigen Zeile, die den von der Funktion `decorate` zurückgegebenen Wert enthält.

Einschränkungen

- $1 \leq N \leq 5000$.
- $0 \leq A_i \leq 1\,000\,000\,000$.

Punktevergabe

Dein Programm wird anhand einer Reihe von Testfällen getestet, die nach Teilaufgaben gruppiert sind. Um die Punktzahl zu erhalten, die einer Teilaufgabe zugeordnet ist, musst du alle darin enthaltenen Testfälle korrekt lösen.

- **Teilaufgabe 0 [0 Punkte]:** Beispiel-Testfälle.
- **Teilaufgabe 1 [12 Punkte]:** $N \leq 1500$, A ist nicht abnehmend (d.h. für $i < j$, $A_i \leq A_j$).
- **Teilaufgabe 2 [4 Punkte]:** $N \leq 10$, $A_i \leq 1000$
- **Teilaufgabe 3 [22 Punkte]:** $N \leq 250$
- **Teilaufgabe 4 [26 Punkte]:** $N \leq 800$
- **Teilaufgabe 5 [33 Punkte]:** $N \leq 1500$
- **Teilaufgabe 6 [3 Punkte]:** Keine zusätzlichen Einschränkungen.

Beispiele

stdin	stdout
5 0 1 2 3 4	3
5 4 3 1 1 5	3

Erklärung

Im ersten Beispiel kann gezeigt werden, dass folgende Zugfolge optimal ist:

- Erster Zug des Senats: Das einzige Subarray, welches den schlechtest möglichen Ausgang für ihn maximiert: $[2, 3, 4]$.
- Antwort der Plebianer: Aus $[2, 3, 4]$ müssen sie ein Subarray mit einer Länge von mindestens zwei wählen; die Wahl von $[2, 3]$ minimiert das schlussendliche Resultat.
- Schliesslich wählt der Senat 3.

Im zweiten Beispiel:

- Erster Zug des Senats: Der einzige Eröffnungszug, welcher einen Wert von mehr als 2 garantiert, ist die Wahl des Blocks $[4, 3, 1]$ (Länge 3, mindestens die Hälfte von 5 und nicht die ganze Reihe).
- Antwort der Plebianer: Aus $[4, 3, 1]$ müssen sie einen Block der Länge 2 behalten; die Wahl von $[3, 1]$ minimiert das schlussendliche Resultat.
- Schliesslich wählt der Senat 3.

Jeder andere erste Zug (z.B. $[3, 1, 1]$, $[1, 1, 5]$ oder ein Block der Länge 4) lässt die Plebianer den Ausgang auf 1 hinunterzwingen, also endet optimales Spielen beider Seiten mit dem Preis 3.